

Übung 2: Einrichtung einer CI/CD-Pipeline

Team 6 (Thema E-Vote) - Abgabe:

Link: https://github.com/jacque-schu/MSE_eVote

Gemeinsam als Team haben wir uns über Discord zum Thema „Einrichtung einer CI/CD-Pipeline“ ausgetauscht. Für die erste folgende Aufgabe haben wir in einem Google-Docs Dokument unsere Antworten zusammengetragen.

Aufgabe 1: Einführung in CI/CD

Diskutieren Sie im Team (a) die grundlegenden Konzepte von Continuous Integration und Continuous Deployment und (b) wie CI/CD Ihre Softwareentwicklung unterstützt und wie automatisierte Tests und Builds in den Workflow integriert werden können. Halten Sie Ihre Diskussionsresultate fest, um diese in der Zwischenpräsentation zu vorzustellen.

Lösungen aus unserem Google-Docs Dokument:

1. Was ist CI/CD und warum wird es genutzt? – erarbeitet von Fabian Tom Scholl

CI/CD steht für Continuous Integration (kontinuierliche Integration) und Continuous Deployment/Delivery (kontinuierliche Bereitstellung/Auslieferung). Es handelt sich um ein Konzept aus der Softwareentwicklung, das darauf abzielt, den Prozess von der Codeerstellung bis zur Auslieferung an den Nutzer möglichst automatisiert, effizient und fehlerarm zu gestalten.

1.1 Continuous Integration (CI)

Bedeutet, dass Entwickler ihren Code regelmäßig (oft mehrmals täglich) in ein zentrales Repository integrieren.

Dabei wird automatisch überprüft, ob der neue Code fehlerfrei mit dem bestehenden Code zusammenarbeitet.

Automatische Builds (Kompilierungen) und Tests stellen sicher, dass neue Änderungen keine alten Funktionen zerstören („Regressionen“).

1.2 Continuous Delivery (CD)

- Ziel: Den Code jederzeit bereitstellen zu können.
- Nach der erfolgreichen Integration wird der Code automatisch in eine staging- oder Testumgebung übertragen.

- Die Software ist so stets in einem Zustand, der prinzipiell ausgeliefert werden könnte, aber die Freigabe erfolgt noch manuell.

1.3 Praxisnutzen

1. Schnelle Fehlererkennung
Jeder neue Code wird sofort getestet – Fehler fallen früh auf und sind leichter zu beheben.
2. Automatisierte Abläufe
Vom Build über Tests bis zum Deployment laufen viele Schritte automatisch ab – das spart Zeit und reduziert menschliche Fehler.
3. Hohe Softwarequalität
Durch regelmäßige Tests und Integrationen bleibt der Code stabil und wartbar.
4. Transparenz im Entwicklungsprozess
Alle Teammitglieder sehen den aktuellen Status, Build-Ergebnisse und Testergebnisse.
5. Kürzere Entwicklungszyklen
Neue Features und Bugfixes gelangen schneller zum Nutzer – ein klarer Wettbewerbsvorteil.

2. Continuous Integration im Detail – erarbeitet von Fabian Meintzer

Continuous Integration (CI) sorgt dafür, dass neue Codeänderungen kontinuierlich in das zentrale Repository integriert und automatisch überprüft werden. Ziel ist es, frühzeitig Fehler zu erkennen, die Softwarequalität hochzuhalten und den Entwicklungsprozess durch automatisierte Abläufe effizienter zu gestalten.

2.1 Wie funktioniert CI technisch?

Technisch basiert Continuous Integration auf einem automatisierten Prozess, der bei jedem Code-Commit ausgelöst wird. Der CI-Server (z.B. Jenkins, GitHub Actions, GitLab CI) erkennt Änderungen im Repository, lädt den aktuellen Code und führt automatisch Build- und Testprozesse aus. Dadurch wird überprüft, ob der neue Code korrekt kompiliert und mit dem bestehenden Code kompatibel ist. Die Ergebnisse werden anschließend an das Team gemeldet, um Fehler frühzeitig sichtbar zu machen.

2.2 Beispiele für typische Aufgaben in der CI-Phase

- automatisierte Builds: Das Projekt wird nach jedem Commit automatisch kompiliert und auf Fehler geprüft
- Unit-Tests: Einzelne Funktionen und Methoden werden automatisiert getestet.
- Integrationstest: Das Zusammenspiel verschiedener Softwaremodule wird überprüft.

- Statische Codeanalyse: Der Code wird automatisch auf Stil- und Qualitätsprobleme untersucht (z.B. mit SonarQube oder ESLint)
- Sicherheitsüberprüfungen: Der Code wird automatisiert auf bekannte Sicherheitslücken analysiert.
- Code Coverage: Es wird gemessen, welcher Anteil des Codes von Tests erfasst ist.
- Erstellung von Berichten: Zeigt Entwicklern, wo Fehler oder Qualitätsprobleme liegen

3. Continuous Deployment/Delivery im Detail – erarbeitet von Heike Fritz

Continuous Delivery (CD) und Continuous Deployment (CDp) sind zentrale Praktiken im DevOps-Kontext, die den Prozess der Softwarebereitstellung automatisieren und beschleunigen. Sie zielen darauf ab, Codeänderungen effizient, sicher und kontinuierlich in produktionsreife Zustände zu überführen.

3.1 Unterschied zwischen Delivery und Deployment

Continuous Delivery beschreibt die Phase, in der Softwareänderungen nach jedem erfolgreichen Build und Test automatisch in eine Staging-Umgebung (z. B. Testserver) übertragen werden. Diese Versionen sind stets bereit für die Produktion, werden jedoch erst nach manueller Freigabe veröffentlicht. Continuous Deployment hingegen geht einen Schritt weiter: Jede Änderung, die alle automatisierten Tests besteht, wird ohne manuelle Eingriffe direkt in die Produktionsumgebung ausgerollt. Der entscheidende Unterschied liegt somit im Grad der Automatisierung – Delivery erfordert menschliche Freigabe, Deployment ist vollständig automatisiert.

3.2 Ablauf der automatisierten Auslieferung

Der typische Ablauf innerhalb einer CI/CD-Pipeline beginnt mit der Continuous Integration (CI), die Codeänderungen automatisch baut und testet. Anschließend übernimmt Continuous Delivery, welche die getesteten Builds in eine stagingfähige Umgebung überführt. Dort werden weitere Tests (z. B. Integration, UI, Last) durchgeführt. Beim Continuous Deployment folgt unmittelbar der automatische Rollout in Produktion, sobald alle Prüfungen bestanden sind. Tools wie Jenkins, GitHub Actions oder GitLab CI orchestrieren diesen Prozess durch definierte Pipelines.

3.3 Nutzen der Automatisierung

Automatisierte Auslieferung bietet mehrere Vorteile. Sie reduziert das Risiko menschlicher Fehler und steigert die Bereitstellungsgeschwindigkeit erheblich. Da Änderungen in kleineren, häufigeren Schritten ausgeliefert werden, sind Fehler leichter zu erkennen und zu beheben. Organisationen profitieren zudem von einer schnelleren Feedbackschleife – Benutzerreaktionen auf neue Funktionen können

zeitnah analysiert und in weitere Optimierungen überführt werden. Schließlich führt die kontinuierliche Bereitstellung zu höherer Softwarequalität, geringeren Ausfallzeiten und einer deutlich verbesserten Zusammenarbeit zwischen Entwicklungs- und Betriebsteams.

4. Automatisierte Tests in der CI/CD-Pipeline – arbeitet von Mareike Starker

4.1 Welche Rolle spielen Tests?

Automatisierte Tests prüfen, ob der neue Code funktioniert und keine bestehenden Funktionen beeinträchtigt. Sie tragen also zur Qualitätssicherung und Effizienzsteigerung in der Softwareentwicklung bei.

Die Tests werden bei jedem Commit-Vorgang oder Push-/Pull-Vorgang ausgeführt und ermöglichen so eine frühzeitige Fehlererkennung.

4.2 Welche Arten von Tests können integriert werden?

In einer CI/CD-Pipeline werden in der Regel verschiedene automatisierte Tests eingesetzt, um die Qualität, Stabilität und Sicherheit der Software sicherzustellen.

Beispiele von Testarten, die integriert werden:

Nach jedem Commit:

Unit-Tests: Sie testen einzelne Funktionen oder Methoden isoliert. Sie sind schnell und werden direkt nach dem Code-Commit ausgeführt.

Integrations-Tests: Sie überprüfen das Zusammenspiel mehrerer Komponenten oder Module. Sie stellen sicher, dass Schnittstellen korrekt funktionieren.

Smoke-Tests: Das sind schnelle Tests, die prüfen, ob die wichtigsten Funktionen nach einem Build noch funktionieren. Sie dienen als erste Qualitätskontrolle.

Regressionstests: Sie stellen sicher, dass neue Änderungen keine bestehenden Funktionen beeinträchtigen.

Vor dem Deployment:

End-to-End-Tests (E2E): Sie simulieren reale Benutzeraktionen über die gesamte Anwendung hinweg. Sie validieren, ob das System als Ganzes funktioniert.

API-Tests: Sie testen Schnittstellen auf korrekte Antworten, Fehlerbehandlung und Performance.

Sicherheits-Tests: Sie identifizieren Schwachstellen wie SQL-Injections, XSS oder unsichere Authentifizierungsmechanismen. Sie sind besonders wichtig vor dem Deployment.

Performance-Tests: Sie messen Antwortzeiten, Durchsatz und Systemverhalten unter Last. Sie helfen, Skalierbarkeit und Stabilität zu bewerten.

5. Kurze praktische Umsetzung im Git-Repository – erarbeitet von Jacqueline Schulz

GitHub Actions ist eine Plattform für kontinuierliche Integration und kontinuierliche Auslieferung (CI/CD), mit der sich Build-, Test- und Deployment-Prozesse im Repository automatisieren lassen. Workflows können so eingerichtet werden, dass sie bei jedem Commit, Pull Request oder anderen Repository-Ereignissen automatisch starten und Aufgaben wie das Bauen und Testen von Code ausführen. Die Ausführung erfolgt auf virtuellen Linux-, Windows- oder macOS-Servern von GitHub, wodurch keine eigene Infrastruktur notwendig ist.

5.1 Wie sieht eine minimalistische Pipeline-Konfiguration aus

Im Verzeichnis `.github/workflows` des Repositories wird eine YAML Datei angelegt.

Dieser GitHub Actions Workflow automatisiert das Bauen, Testen und Linting (Codeprüfung) einer Python-Anwendung. Er wird ausgeführt, wenn auf dem Main-Branch gepusht oder ein Pull-Request erstellt wird.

name: python-app.yml (Code)

https://github.com/jacque-schu/MSE_eVote/actions/runs/18571440271/workflow

5.2 Wichtige Hinweise zur Einrichtung (z.B. Runner/Actions, Trigger)

Für die Einrichtung einer GitHub Actions CI/CD-Pipeline sind einige Kernaspekte zu beachten:

- **Runner:** Ein Runner ist die virtuelle Maschine, auf der die Jobs aus deinem Workflow ausgeführt werden. GitHub stellt dir standardmäßig Linux-, Windows- und macOS-Runner kostenfrei zur Verfügung, sodass keine eigene Hardware benötigt wird. Optional können auch eigene (self-hosted) Runner in deiner Infrastruktur installiert werden – dies ist aber für Standardanwendungen meist nicht nötig.
- **Actions:** Actions sind wiederverwendbare, vorgefertigte Bausteine für einzelne Schritte eines Workflows (z. B. Checkout, Python-Setup, Tests). Viele nützliche Actions sind direkt im Marketplace verfügbar. Im Workflow werden die Actions einfach durch kurze YAML-Befehle eingebunden. So lassen sich häufige Aufgaben wie das Auschecken des Repos, das Einrichten von Programmiersprachen oder Sicherheitschecks schnell automatisieren.
- **Trigger:** Die Pipeline („Workflow“) wird durch definierte Trigger aktiviert – zum Beispiel bei jedem Commit (push) oder Pull Request (pull_request) auf bestimmte Branches (wie main oder develop). Diese Einstellungen werden im Workflow-YAML über das Feld `on:` festgelegt und ermöglichen eine flexible, gezielte Automatisierung deiner Build- und Testprozesse.

Ein korrekt eingerichteter Workflow sorgt dafür, dass sich die Automatisierung nahtlos in den Entwicklungsprozess integriert, stets auf gesicherten GitHub-Servern ausgeführt wird und jeder Fehler oder Erfolg transparent im Bereich „Actions“ des Repositories nachverfolgt werden kann.

Aufgabe 2: Einrichtung der CI/CD-Pipeline,
Aufgabe 3: Tests hinzufügen,
Aufgabe 4: Deployment-Konzepte,
Aufgabe 5: Branching und Pull Requests in Verbindung mit CI/CD:

siehe Link: https://github.com/jacque-schu/MSE_eVote

Resultat:

Im GitHub Repository wurde eine automatisierte CI-Pipeline eingerichtet und automatisierte Tests integriert. Dieses Dokument sowie das Repository dokumentieren den CI/CD-Prozess und gemeinsam als Gruppe haben wir gelernt, wie dieser mit Git-Workflows (Branches, Pull Requests) kombiniert wird.

All workflows

Filter workflow runs

Showing runs from all workflows

9 workflow runs

Event

Status

Branch

Actor

✓

Add Kommentare Schritt 4

Python application #9: Commit [6ed4122](#) pushed by [fabtomsch](#)

main

1 minute ago

20s

...

✓

Add test

Python application #8: Commit [27614a1](#) pushed by [fabtomsch](#)

main

3 minutes ago

19s

...

✓

Kommentare für python-app.yml

Python application #7: Pull request #10 synchronize by [fabtomsch](#)

fabtomsch

11 minutes ago

56s

...

✗

Kommentare für python-app.yml

Python application #6: Pull request #10 synchronize by [fabtomsch](#)

fabtomsch

15 minutes ago

18s

...

✗

Kommentare für python-app.yml

Python application #5: Pull request #10 synchronize by [fabtomsch](#)

fabtomsch

19 minutes ago

10s

...

✗

Test Error Behebung python-app.yml

Python application #4: Commit [a3a4802](#) pushed by [fabtomsch](#)

fabtomsch

20 minutes ago

Failure

...

✗

Error Behebung python-app.yml

Python application #3: Commit [803f236](#) pushed by [fabtomsch](#)

fabtomsch

24 minutes ago

Failure

...